

实验名称 实验2 两两序列比对算法

一、实验内容及结果

1、调试所给的 Needleman-Wunsch 算法，使其能够正确比对两个序列，利用调试后的 Needleman-Wunsch 算法对以下两条序列进行比对,输出比对结果。

GAAGGTTCCCGCCGCTATCGTAT
AAGGTTCCCGAACCGCTATTATC

源代码：

```
import itertools
import copy
import numpy as np
# seq1='ACACACTA'
# seq2='AGCACACA'
seq1='GAAGGTTCCCGCCGCTATCGTAT'
seq2='AAGGTTCCCGAACCGCTATTATC'

score_dic={'match':1, 'mismatch':0, 'gap':-1}
score_matrix=[[0 for column in range(len(seq1)+1)] for row in range(len(seq2)+1)]
trace_back=[[] for column in range(len(seq1)+1)] for row in range (len(seq2)+1)]
for i in range(len(score_matrix)):
    score_matrix[0][i]=i * (-1)
    if i>0:
        trace_back[0][i].append('left')
for i in range(len(score_matrix)):
    score_matrix[i][0]=i * -1
    if i>0:
        trace_back[i][0].append('up')
trace_back[0][0].append('done')
```

改动：初始化时添加全为 0 的一行一列，所以 score_matrix 和 trace_back 大小应为 len(seq2)+1 而不是 len(seq2)

```

for i in range(1,len(score_matrix)):
    # print(seq2[i-1])
    for j in range(1,len(score_matrix[0])):
        # print(seq1[j-1])
        if seq1[j-1]==seq2[i-1]:
            char_score=score_dic['match']
        else:
            char_score=score_dic['mismatch']
            top_score=score_matrix[i-1][j]+score_dic['gap']
            left_score=score_matrix[i][j-1]+score_dic['gap']
            diag_score=score_matrix[i-1][j-1]+char_score
            score=max(top_score,left_score,diag_score)
            score_matrix[i][j]=score
            if top_score==score:
                trace_back[i][j].append('up')
            if left_score==score:
                trace_back[i][j].append('left')
            if diag_score==score:
                trace_back[i][j].append('diag')

for row in score_matrix:
    print(row)

```

```

pointer=[len(score_matrix), len(score_matrix[0])]
print(pointer)
align_seq1=[]
align_seq2=[]
arrow=trace_back[pointer[0]-1][pointer[1]-1]

```

改动：矩阵维数改变，相应下标随之变化。

seq1[j]=seq2[i]改为 seq1[j-1]=seq2[i-1];

trace_back[pointer[0]][pointer[1]] 改 为 trace_back[pointer[0]-1][pointer[1]-1]

```

def seq_letter_finder(current_arrow, current_pointer):
    if current_arrow == 'diag':
        letter = [seq1[current_pointer[1]-2], seq2[current_pointer[0]-2]]
        next_pointer = [current_pointer[0]-1, current_pointer[1]-1]
        next_arrow = trace_back[next_pointer[0]-1][next_pointer[1]-1]
        return letter, next_arrow, next_pointer

    elif current_arrow == 'left':
        letter = [seq1[current_pointer[1]-2], '-']
        next_pointer = [current_pointer[0], current_pointer[1]-1]
        next_arrow = trace_back[next_pointer[0]-1][next_pointer[1]-1]
        return letter, next_arrow, next_pointer

    else:
        letter = ['-', seq2[current_pointer[0]-2]]
        next_pointer = [current_pointer[0]-1, current_pointer[1]]
        next_arrow = trace_back[next_pointer[0]-1][next_pointer[1]-1]
        return letter, next_arrow, next_pointer

```

改动：不同 arrow 下 letter 的下标变化。

```

def align_seq_finder(rec_arrow, rec_pointer, rec_ls):
    if rec_arrow[0] == 'done':
        rec_ls = [rec_ls[0][::-1], rec_ls[1][::-1]]
        return rec_ls
    else:
        if len(rec_arrow) == 1:
            letter, rec_arrow, rec_pointer = seq_letter_finder(rec_arrow[0], rec_pointer)
            rec_ls[0] += letter[0]
            rec_ls[1] += letter[1]
            return align_seq_finder(rec_arrow, rec_pointer, rec_ls)

        elif len(rec_arrow) == 2:
            arrow1 = copy.deepcopy(rec_arrow[0])
            pointer1 = copy.deepcopy(rec_pointer)
            ls1 = copy.deepcopy(rec_ls)
            arrow2 = copy.deepcopy(rec_arrow[1])
            pointer2 = copy.deepcopy(rec_pointer)
            ls2 = copy.deepcopy(rec_ls)
            letter1, arrow1, pointer1 = seq_letter_finder(arrow1, pointer1)
            letter2, arrow2, pointer2 = seq_letter_finder(arrow2, pointer2)
            ls1[0] += letter1[0]
            ls1[1] += letter1[1]
            ls1[0] += letter2[0]
            ls1[1] += letter2[1]
            return list(itertools.chain(align_seq_finder(arrow1, pointer1, ls1),
                                         align_seq_finder(arrow2, pointer2, ls2)))

```

```

else:
    arrow1 = copy.deepcopy(rec_arrow[0])
    pointer1 = copy.deepcopy(rec_pointer)
    pointer2 = copy.deepcopy(rec_pointer)
    pointer3 = copy.deepcopy(rec_pointer)
    ls1 = copy.deepcopy(rec_ls)
    ls2 = copy.deepcopy(rec_ls)
    ls3 = copy.deepcopy(rec_ls)
    letter, arrow1, pointer1 = seq_letter_finder(arrow1, pointer1)
    ls1[0] += letter[0]
    ls1[1] += letter[1]
    arrow2 = rec_arrow[1]
    letter, arrow2, pointer2 = seq_letter_finder(arrow2, pointer2)
    ls2[0] += letter[0]
    ls2[1] += letter[1]
    arrow3 = rec_arrow[2]
    letter, arrow3, pointer3 = seq_letter_finder(arrow3, pointer3)
    ls3[0] += letter[0]
    ls3[1] += letter[1]
    return list(itertools.chain(align_seq_finder(arrow1, pointer1, ls1),
                                align_seq_finder(arrow2, pointer2, ls2),
                                align_seq_finder(arrow3, pointer3, ls3)))

```

```

ls=align_seq_finder(arrow,pointer,[' ',' '])
for i in range(0,len(ls),2):
    print(ls[i])
    print(ls[i+1])
    print()

```

改动：逐行打印结果，便于序列比对；不同的比对方法间用空行隔开。

运行结果 (截图):

得分矩阵:

```
[0, -1, -2, -3, -4, -5, -6, -7, -8, -9, -10, -11, -12, -13, -14, -15, -16, -17, -18, -19, -20, -21, -22, -23]
[-1, 0, 0, -1, -2, -3, -4, -5, -6, -7, -8, -9, -10, -11, -12, -13, -14, -15, -16, -17, -18, -19, -20, -21]
[-2, -1, 1, 1, 0, -1, -2, -3, -4, -5, -6, -7, -8, -9, -10, -11, -12, -13, -14, -15, -16, -17, -18, -19]
[-3, -1, 0, 1, 2, 1, 0, -1, -2, -3, -4, -5, -6, -7, -8, -9, -10, -11, -12, -13, -14, -15, -16, -17]
[-4, -2, -1, 0, 2, 3, 2, 1, 0, -1, -2, -3, -4, -5, -6, -7, -8, -9, -10, -11, -12, -13, -14, -15]
[-5, -3, -2, -1, 1, 2, 4, 3, 2, 1, 0, -1, -2, -3, -4, -5, -6, -7, -8, -9, -10, -11, -12, -13]
[-6, -4, -3, -2, 0, 1, 3, 5, 4, 3, 2, 1, 0, -1, -2, -3, -4, -5, -6, -7, -8, -9, -10, -11]
[-7, -5, -4, -3, -1, 0, 2, 4, 6, 5, 4, 3, 2, 1, 0, -1, -2, -3, -4, -5, -6, -7, -8, -9]
[-8, -6, -5, -4, -2, -1, 1, 3, 5, 7, 6, 5, 4, 3, 2, 1, 0, -1, -2, -3, -4, -5, -6, -7]
[-9, -7, -6, -5, -3, -2, 0, 2, 4, 6, 8, 7, 6, 5, 4, 3, 2, 1, 0, -1, -2, -3, -4, -5]
[-10, -8, -7, -6, -4, -2, -1, 1, 3, 5, 7, 9, 8, 7, 6, 5, 4, 3, 2, 1, 0, -1, -2, -3]
[-11, -9, -7, -6, -5, -3, -2, 0, 2, 4, 6, 8, 9, 8, 7, 6, 5, 5, 4, 3, 2, 1, 0, -1]
[-12, -10, -8, -6, -6, -4, -3, -1, 1, 3, 5, 7, 8, 9, 8, 7, 6, 6, 5, 4, 3, 2, 2, 1]
[-13, -11, -9, -7, -6, -5, -4, -2, 0, 2, 4, 6, 8, 9, 9, 8, 7, 6, 6, 5, 4, 3, 2]
[-14, -12, -10, -8, -7, -6, -5, -3, -1, 1, 3, 5, 7, 9, 9, 10, 9, 8, 7, 7, 6, 5, 4, 3]
[-15, -13, -11, -9, -7, -6, -6, -4, -2, 0, 2, 4, 6, 8, 10, 9, 10, 9, 8, 7, 8, 7, 6, 5]
[-16, -14, -12, -10, -8, -7, -6, -5, -3, -1, 1, 3, 5, 7, 9, 11, 10, 10, 9, 9, 8, 8, 7, 6]
[-17, -15, -13, -11, -9, -8, -6, -5, -4, -2, 0, 2, 4, 6, 8, 10, 12, 11, 11, 10, 9, 9, 8, 8]
[-18, -16, -14, -12, -10, -9, -7, -6, -5, -3, -1, 1, 3, 5, 7, 9, 11, 13, 12, 11, 10, 9, 10, 9]
[-19, -17, -15, -13, -11, -10, -8, -6, -6, -4, -2, 0, 2, 4, 6, 8, 10, 12, 14, 13, 12, 11, 10, 11]
[-20, -18, -16, -14, -12, -11, -9, -7, -6, -5, -3, -1, 1, 3, 5, 7, 9, 11, 13, 14, 13, 13, 12, 11]
[-21, -19, -17, -15, -13, -12, -10, -8, -7, -6, -4, -2, 0, 2, 4, 6, 8, 10, 12, 13, 14, 13, 14, 13]
[-22, -20, -18, -16, -14, -13, -11, -9, -8, -7, -5, -3, -1, 1, 3, 5, 7, 9, 11, 12, 13, 15, 14, 15]
[-23, -21, -19, -17, -15, -14, -12, -10, -8, -7, -6, -4, -2, 0, 2, 4, 6, 8, 10, 12, 12, 14, 15, 14]
```

序列比对结果:

```
GAAGGTTCCCG--CCGCTATCGTAT-
-AAGGTTCCCGAACCGCTAT--TATC
```

```
GAAGGTTCCCG--CCGCTATCGTAT
-AAGGTTCCCGAACCGCTATTATC-
```

```
GAAGGTTCCCG--CCGCTATCGTAT
-AAGGTTCCCGAACCGCTATTAT-C
```